COMP 110

Recursive Structures

First, a review of recursion…

- So far, we've only covered recursive *functions*
- Calling a function within itself, typically with a smaller input.
- Two components:
  - Base case(s)
    - Where recursion *ends*
    - Often smallest input(s)
    - Prevent infinite loops!
  - Recursive Rule
    - Definition to handle all inputs that aren't base case.
    - Expresses function in terms of smaller calls to the function.
      - (e.g. expressing f(n) in terms of f(n-1))

In Python

```python
def f(n: int) -> int:
    if n == 0: # base case
        return 0
    else: # recursive rule
        return 1 + f(n-1)

f(2)
```

Recursive function checklist:

Base case:

- ❑ Does the function have a clear base case?
  - ❑ Ensure the base case returns a result directly (without calling the function again).
- ❑ Will the base case *always* be reached?

Recursive case:

- ❑ Does the function have a recursive case that *progresses toward the base case*?
  - ❑ Does the recursive call have the right arguments? The function should call itself on a simpler or smaller version of the problem.
- ❑ Have you tested your function with multiple cases, including edge cases?

Stack Overflow and Recursion Errors

When a programmer writes a function that calls itself indefinitely (*infinitely*), the **function call stack** will *overflow…*

This leads to a **`Stack Overflow`** or **`Recursion Error`**:

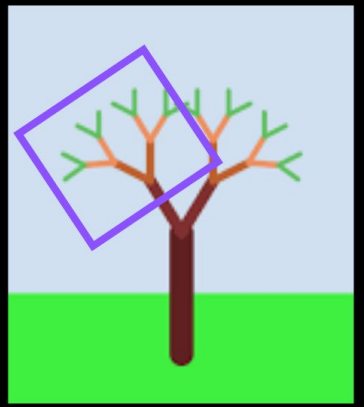
```
RecursionError: maximum recursion depth exceeded while
calling a Python object
```

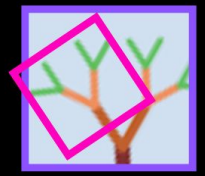
Recursion: defining an operation/object in terms of itself

Recursion doesn't only occur in functions!

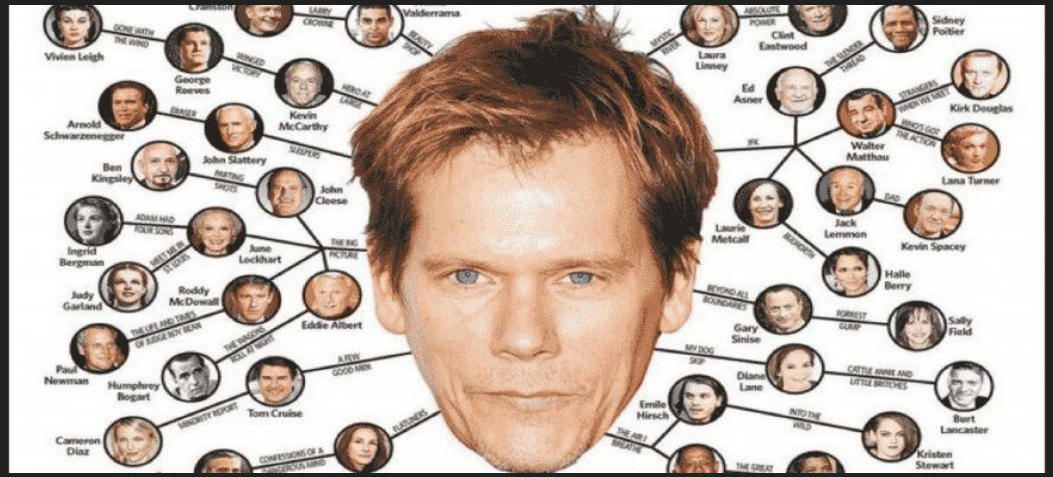
A real-world phenomenon! Examples:

- **You** have **parents**, who have **parents**, who have **parents**, who have **parents**, who…
  … were **early humans**
- A **tree** has **branches**, which have **branches**, which have **branches**, which…
  … have **leaves**



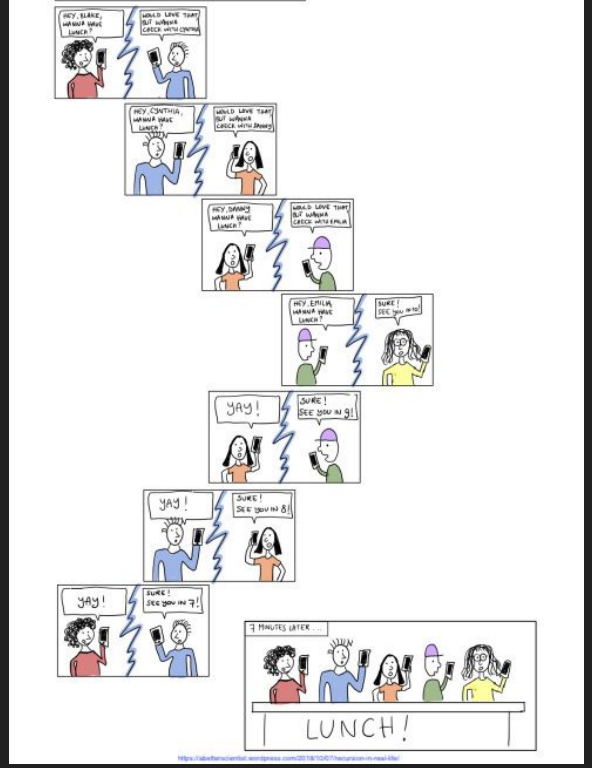


Different recursive structures for different purposes



Six degrees of Kevin Bacon

graph/network



Coordinating plans with individual phone calls

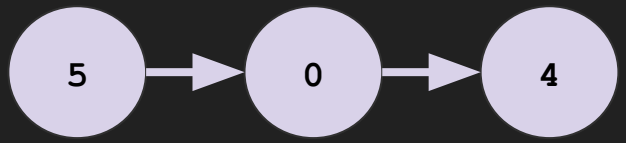
linked list

Anatomy of a Singly-Linked List

Classic List:

Has values at indices

Linked List:



Made of “nodes” that have values and pointers to next node

Anatomy of a Singly-Linked List

Classic List:

5	0	4

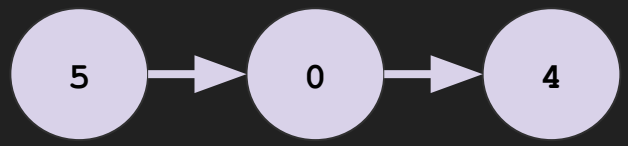
Has values at indices

Anatomy of a Singly-Linked List

Benefits of a linked list:

- Allow more flexibility in memory allocation
- “Easier” to add/remove items

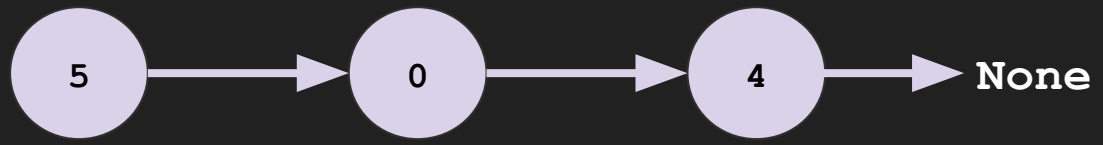
Linked List:



Made of “nodes” that have values and pointers to next node

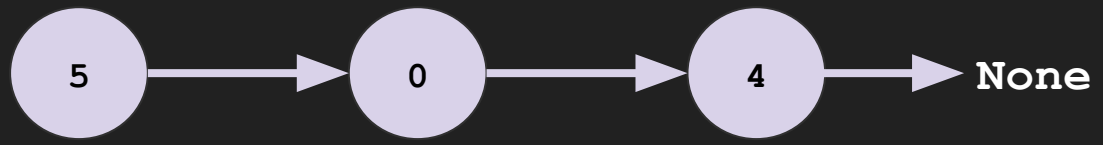
Anatomy of a Singly-Linked List

Node	
value	(int)
next	(Node)



Anatomy of a Singly-Linked List

Node	
value	(int)
next	(Node)

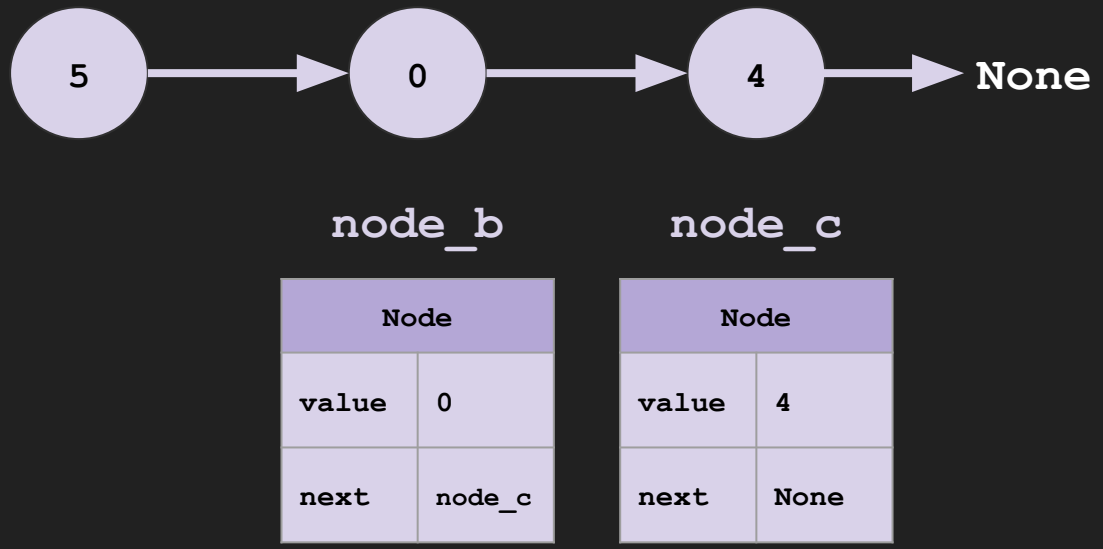


node_c

Node	
value	4
next	None

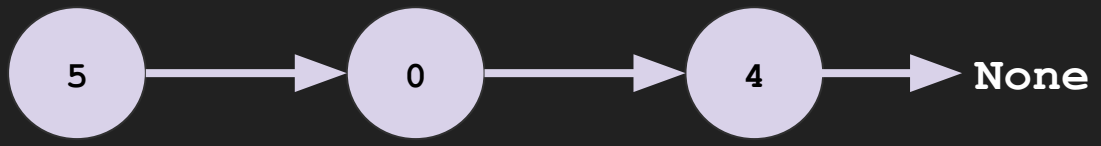
Anatomy of a Singly-Linked List

Node	
value	(int)
next	(Node)



Anatomy of a Singly-Linked List

Node	
value	(int)
next	(Node)



node_a

Node	
value	5
next	node_b

node_b

Node	
value	0
next	node_c

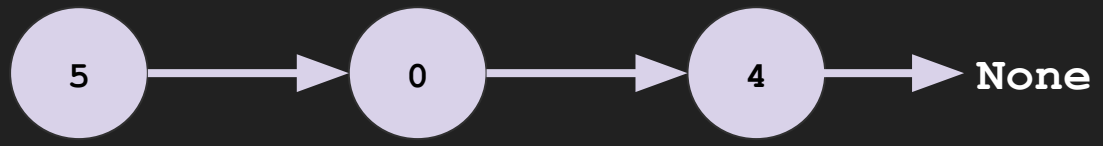
node_c

Node	
value	4
next	None

Anatomy of a Singly-Linked List

Reference it by referencing the "front" node.

`print(node_a)` should print the whole linked list.



node_a

Node	
value	5
next	node_b

node_b

Node	
value	0
next	node_c

node_c

Node	
value	4
next	None

Let’s Build this in Python!

- In your workspace in your “exercises” folder, create a folder named “ex08”
- Create a file called linked_list.py

Mini-lesson: Union Types

- Variables can take more than one different type.
- Denoted using |
- In a function signature:
  - ```
    def add(a: int | float, b: int | float) -> int | float:
        return a + b
    ```
- In an attribute declaration:
  - ```
    next: Node | None
    ```

Memory diagram

```python
from __future__ import annotations

class Node:

    value: int
    next: Node | None

    def __init__(self, value: int, next: Node | None):
        self.value = value
        self.next = next

node_c: Node = Node(4, None)
node_b: Node = Node(0, node_c)
```

Memory diagram

```python
from __future__ import annotations

class Node:

    value: int
    next: Node | None

    def __init__(self, value: int, next: Node | None):
        self.value = value
        self.next = next

node_c: Node = Node(4, None)
node_b: Node = Node(0, node_c)
```

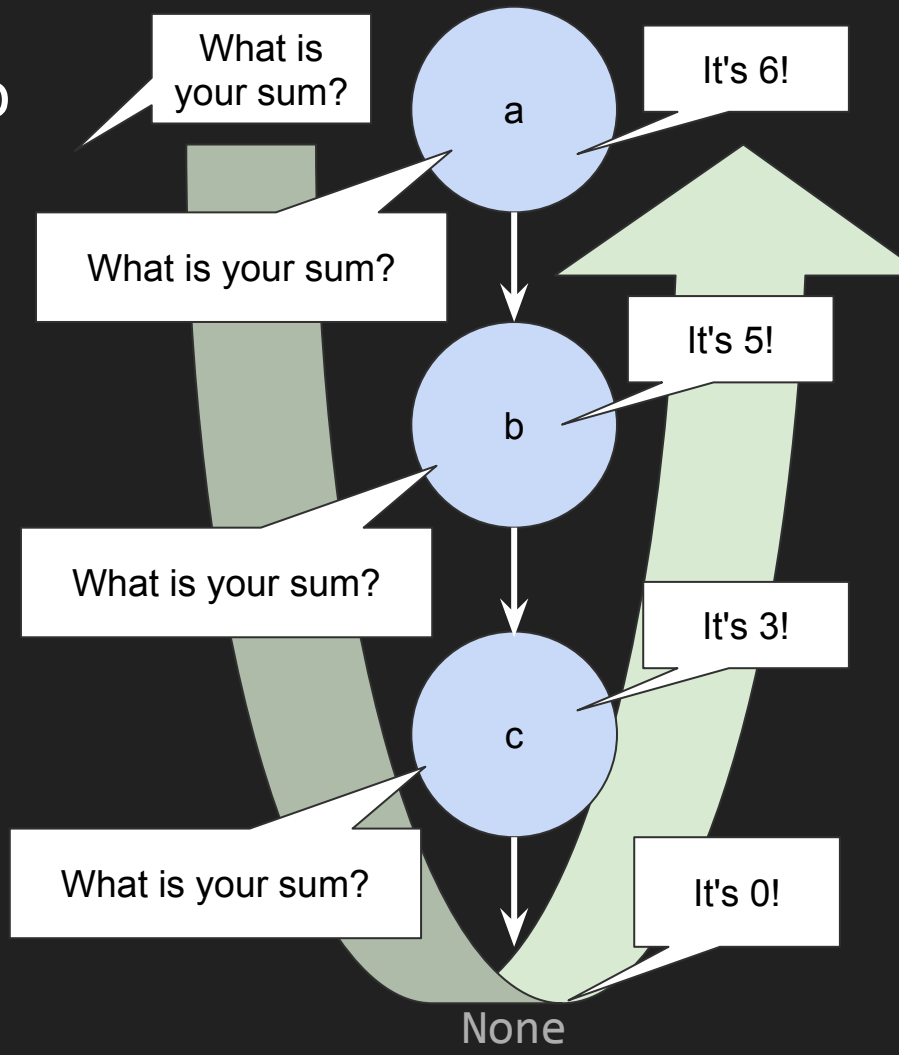
Let's write a <u>function</u> called **`sum`** that adds up the **`value`**s of all **`Node`**s in the linked list.

A Recursive `sum` Algorithm Demo

1. When you are asked,
   "what is your sum?"

2. Ask the ***next*** Node,
   "what is your sum?"
   Wait patiently for an answer!

3. Once the answer is returned back to you, add ***your value to it***, then turn to the person who asked you and give them this answer.



For reference: recursive function checklist:

Base case:

- ❏ Does the function have a clear base case?
  - ❏ Ensure the base case returns a result directly (without calling the function again).
- ❏ Will the base case *always* be reached?

Recursive case:

- ❏ Does the function have a recursive case that *progresses toward the base case*?
  - ❏ Does the recursive call have the right arguments? The function should call itself on a simpler or smaller version of the problem.
- ❏ Have you tested your function with multiple cases, including edge cases?

(For practice)

Try diagramming the **`sum`** function call!

```python
from __future__ import annotations

class Node:

    value: int
    next: Node | None

    def __init__(self, value: int, next: Node | None):
        self.value = value
        self.next = next


def sum(head: Node | None) -> int:
    if head is None:
        return 0
    else:
        return head.value + sum(head.next)

node_c: Node = Node(4, None)
node_b: Node = Node(0, node_c)
print(sum(node_b))
```

Your turn: Memory Diagram

```python
from __future__ import annotations

class Node:
    """Node in a singly-linked list recursive structure."""
    value: int
    next: Node | None

    def __init__(self, value: int, next: Node | None):
        self.value = value
        self.next = next

    def __str__(self) -> str:
        if self.next is None:
            return f"{self.value} -> None"
        else:
            return f"{self.value} -> {self.next}"

courses: Node = Node(110, Node(210, None))
print(courses)
```

and discuss with a neighbor:

1. What does the __str__ method do?
2. Is this method recursive? How do we know?

Memory Diagram

```python
from __future__ import annotations

class Node:
    """Node in a singly-linked list recursive structure."""
    value: int
    next: Node | None

    def __init__(self, value: int, next: Node | None):
        self.value = value
        self.next = next

    def __str__(self) -> str:
        if self.next is None:
            return f"{self.value} -> None"
        else:
            return f"{self.value} -> {self.next}"

courses: Node = Node(110, Node(210, None))
print(courses)
```

Memory Diagram

```python
from __future__ import annotations

class Node:
    """Node in a singly-linked list recursive structure."""
    value: int
    next: Node | None

    def __init__(self, value: int, next: Node | None):
        self.value = value
        self.next = next

    def __str__(self) -> str:
        if self.next is None:
            return f"{self.value} -> None"
        else:
            return f"{self.value} -> {self.next}"

courses: Node = Node(110, Node(210, None))
print(courses)
```

Insert this method into your code!